

Learning and Evaluation on the IVM Lattice

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Learning and Evaluation on the IVM Lattice	1
1.1 Overview	1
1.2 Why held-out evaluation is harder on a lattice	1
1.3 K-fold site splits	2
1.4 Cross-validation and honest model selection	2
1.5 Temporal splits for dynamics identification	2
1.6 Learning curves	3
1.7 Three-way splitting and trainable site fits	3
1.8 Verification	4
1.9 Cross-References	4

1 Learning and Evaluation on the IVM Lattice

1.1 Overview

Learning on a lattice is easy to fake. A learner that merely interpolates its observations through the graph can look excellent on the very data it was fit to, and a dynamics model identified on early snapshots can silently diverge on late ones. This section develops the training/testing methodology used to evaluate the two IVM learners honestly — the static field learner of **Static IVM Field Learning** and the dynamics identifier of **IVM Lattice Dynamics**. All utilities live in the `learning_eval.py` module (`kfold_site_splits()`, `cross_validate_field()`, `trajectory_train_test()`, `learning_curve()`, `enclosing_radius()`), use `numpy` only, and are fully deterministic for fixed seeds. Every behavioral claim below is pinned in `tests/test_learning_eval.py` with fixed RNG seeds and real numerics.

1.2 Why held-out evaluation is harder on a lattice

On independent data, random resampling is benign. On a lattice it is not: field values at neighboring sites are strongly correlated, and the learners of `ivm_field.py` exploit exactly that correlation — the kernel-weighted target of the normal equations $(??)$ predicts an unobserved site as a weighted average of its graph neighbors. A random split that leaves graph neighbors on both sides therefore lets the learner “cheat”: the held-out site is predicted from observations one hop away, and the apparent skill measures the smoothness of the field under $(??)$, not generalization to unseen territory. The autocorrelation that makes lattice learning work is the same autocorrelation that inflates its evaluation:

$$\text{Cov}(f_i, f_j) \approx \sigma_f^2 K(d(i, j)), \quad d(i, j) = \text{hop distance on the IVM graph}, \quad (1)$$

with K the Gaussian hop kernel of (??) and σ_f^2 the field variance. Because K decays with hop distance, the leakage is local — but the IVM ball is small, and a random split leaks everywhere.

The methodology below therefore always holds out *structure*, not just rows: whole folds of sites for the field learner (never shared between training and scoring), and a temporal suffix of snapshots for dynamics identification (never shuffled). The pinned numerical result worth internalizing: on the radius-2 ball (55 sites, 80% observed), the noise-free held-out error of the kernel-weighted interpolator is dominated not by observation noise but by *coverage geometry* — which sites happen to be observed — so per-fold scores vary substantially even at fixed sample size, and honest evaluation must average over folds.

1.3 K-fold site splits

`kfold_site_splits(observed_sites, k, seed)` partitions observed sites into k folds by a seeded `numpy.random.default_rng` permutation cut with `numpy.array_split` (the first $n \bmod k$ folds receive one extra site). Each split is a (train, test) pair; folds are pairwise disjoint and jointly exhaustive, and site order within each list follows the original normalized index order, so the splits are a pure function of $(\text{sites}, k, \text{seed})$. With s the seeded permutation and F_j fold j 's index set, the held-out score of a fit $\hat{f}_\lambda^{(-F_j)}$ that never saw F_j is

$$\widehat{M}(\lambda, F_j) = \frac{1}{|F_j|} \sum_{i \in F_j} (\hat{f}_\lambda^{(-F_j)}(i) - y_i)^2, \quad (2)$$

the quantity `IVMField.score()` returns. Duplicate sites are rejected after normalization (two projective representatives of one lattice point would sit on both sides of a split), as is $k < 2$ or $k > n$.

1.4 Cross-validation and honest model selection

`cross_validate_field(field_values, sites, k, lam_grid, seed)` runs (2) for every fold and every regularization candidate of `lam_grid`, fitting each fold from scratch on the training sites only — a fresh `IVMField.lattice_ball` per fit, via `IVMField.learn()` with the objective (??). The result is the full table $\widehat{M}(\lambda, F_j)$ with per-candidate mean and standard deviation over folds, and the selected strength

$$\lambda^* = \arg \min_{\lambda \in \mathcal{G}} \frac{1}{k} \sum_{j=1}^k \widehat{M}(\lambda, F_j), \quad (3)$$

resolved to the earliest candidate on ties. Selection uses validation folds only; the returned `refit_field` is then refit with λ^* on *all* observed sites, matching the deployment condition where every observation is available. Separating the two is what makes the selection honest: choosing λ on the same data the final model fits is how interpolation gets mistaken for skill.

Two regimes are pinned numerically on synthetic fields. With exact observations of a harmonic (linear in embedded XYZ) field at 80% coverage, cross-validation selects $\lambda^* = 0$: the kernel-weighted interpolator generalizes best, and Laplacian smoothing only biases the fit away from exact data — the mean held-out MSE is strictly increasing in λ across a grid spanning two decades. With noisy observations of a smooth non-harmonic field, the selection flips to $\lambda^* > 0$: interpolation carries observation noise into the fit, and moderate regularization wins on the held-out folds. The margin is honest about the learner's limits — because observed rows are pinned to their data, the noise propagates through the graph regardless of λ , so the achievable gain is set by how much neighbor averaging the kernel already performs, not by the regularizer alone.

1.5 Temporal splits for dynamics identification

`trajectory_train_test(observed, split_frac, lattice)` evaluates `fit_trajectory()`-style identification of the coupling α (**IVM Lattice Dynamics**). The snapshot matrix ($T + 1$ rows, row 0 the initial condition) is split **temporally**:

the first nearest-integer-rounded fraction of rows trains, the trailing rows are held out, and no shuffling is ever applied — a shuffled split interpolates between adjacent snapshots of a deterministic trajectory and hides exactly the forecast error being measured. The split point is clamped so training keeps at least two rows (initial condition plus one snapshot) and testing keeps at least one. Identification uses the training prefix only, with candidate grid and refinement passed through to `fit_trajectory()`. Two held-out errors are reported. The multi-step continuation error re-simulates from the observed initial condition with the identified $\hat{\alpha}$ and scores only the held-out rows:

$$M_{\text{test}}(\hat{\alpha}) = \frac{1}{(T - t_s) N} \sum_{t=t_s+1}^T \|u_{\hat{\alpha}}(t) - u_{\text{obs}}(t)\|^2, \quad (4)$$

where t_s is the last training row. This is the stringent metric: for any model that is not exactly right, error compounds with horizon, so the held-out tail punishes misspecification that early rows forgive. The one-step-ahead error is teacher-forced — each held-out transition is predicted a single step from the *true* previous state, the first transition starting from the last training row:

$$M_1(\hat{\alpha}) = \frac{1}{(T - t_s) N} \sum_{t=t_s}^{T-1} \|\text{step}(u_{\text{obs}}(t); \hat{\alpha}) - u_{\text{obs}}(t+1)\|^2, \quad (5)$$

isolating local prediction error from error accumulation. The contrast between (4) and (5) is diagnostic: a correct model drives both to zero (pinned exactly: a heat trajectory with $\alpha = 0.3$ in the candidate grid yields train, continuation, and one-step errors of exactly 0), while a misspecified model class — a heat model identified on majority-dynamics data — shows a modest training error, a held-out continuation error more than twice as large (horizon compounding), and a one-step error five times smaller than the continuation error (local prediction is easier than long-horizon forecasting). Note the direction of the misspecification: `majority_step()` requires integer states, so the float-continuum heat model can be fit to integer data, but not the reverse.

1.6 Learning curves

`learning_curve(values, sites, train_fracs, seed)` traces the classic data-coverage curve on a lattice. One seeded permutation of the observed sites is drawn; for each requested fraction p the first $m(p) = \text{round}(p \cdot n)$ sites of that permutation (clamped to $[1, n - 1]$) form the training subset and the remainder the held-out complement:

$$M(p) = \frac{1}{n - m(p)} \sum_{i \notin P_{m(p)}} (\hat{f}_{P_{m(p)}}(i) - y_i)^2, \quad P_m = \text{first } m \text{ entries of the permutation}, \quad (6)$$

reported together with the in-sample score of the same fit. The subsets are *nested* by construction — growing p never removes an earlier observation — so the curve is a proper coverage curve rather than a family of independent splits. The pinned example (radius-2 ball, quadratic bowl observed with noise $\sigma = 0.4$ everywhere, $\lambda = 0.05$) shows held-out MSE falling by more than a factor of two from 10% to 85% coverage, with non-monotone wiggles in between: with few observations the held-out complement is large and its geometry varies between fractions, so the curve is monotone-ish, not monotone — the final point sits far below the first, and that is the asserted invariant. The curve also separates interpolation from generalization visibly: the in-sample MSE stays below 0.15 at every coverage level while the held-out MSE never drops below 0.7 — the pinned observations are fit closely at all sizes, and the difficulty is entirely on unseen sites.

1.7 Three-way splitting and trainable site fits

The lattice surfaces above all share one shape: a seeded split of sites or snapshots, a fit on the training side, and scores on the held-out side. The same shape is available for plain tabular work in the same module (`src/quadmath`

/learn/learning_eval.py), under the same discipline — all split randomness confined to the seeded permutation of the split, and fits that are pure deterministic functions of their inputs.

`three_way_split(n, train_frac=0.6, val_frac=0.2, seed=0)` partitions `range(n)` into disjoint train/validation/test index lists. One seeded `numpy.random.default_rng` permutation of `range(n)` is drawn and cut into three contiguous blocks: the first $m_{train} = \text{round}(\text{train_frac} \cdot n)$ indices train, the next $m_{val} = \text{round}(\text{val_frac} \cdot n)$ validate, and the remainder test. Each returned list is sorted, the three are pairwise disjoint with union exactly `range(n)`, and the split is a deterministic function of $(n, \text{train_frac}, \text{val_frac}, \text{seed})$ — the same deterministic-permutation contract as the k-fold site splits above, applied to a plain index range rather than observed lattice sites. Rounding is unguarded at the tails: for small n a trailing block may come back empty ($n = 3$ under the default fractions leaves the test list empty), and a `ValueError` is raised when either fraction is not strictly positive or their sum reaches 1 — a split with no held-out rows at all cannot be honest.

`ridge_site_fit(features, values, lam=1e-3)` fits a single linear site model in closed form. With X_c the mean-centered design matrix, y_c the mean-centered target, and $\bar{x}$, $\bar{y}$ the feature-column means and target mean, the ridge normal equations are solved exactly for the slope vector w :

$$(X_c^\top X_c + \lambda I) w = X_c^\top y_c, \quad b = \bar{y} - \bar{x}^\top w, \quad (7)$$

with b the intercept recovered from the centered solve; $\lambda \geq 0$ penalizes the centered coefficient norm, shrinking w toward zero, and $\lambda = 0$ recovers the exact least-squares solve. The returned `RidgeSiteFit` carries coefficients, intercept, and the in-sample `train_mse`, and the fit is a deterministic function of $(\text{features}, \text{values}, \text{lam})$ — a `numpy.linalg.solve`, not an iterative optimizer, so repeated calls on the same inputs reproduce it bit for bit. Validation is explicit: a negative λ , a non-2-D design matrix, a non-1-D target, disagreeing sample counts, or zero rows each raise before any linear algebra runs.

`GradientDescentTrainer(lr=0.05, max_iters=300, tol=1e-9)` is the iterative counterpart, useful where the closed form is deliberately being compared against: a full-batch gradient-descent fit of the same linear model. `fit` standardizes the feature columns internally (per-column zero mean and unit standard deviation, with constant columns passing through at standard deviation 1), then runs at most `max_iters` full-batch updates on the mean-squared-error loss,

$$\theta \leftarrow \theta - \frac{2\eta}{m} \begin{bmatrix} X_s^\top e \\ \mathbf{1}^\top e \end{bmatrix}, \quad e = X_s w + b - y, \quad (8)$$

where X_s is the standardized design, m the row count, η the learning rate, w and b the coefficient vector and bias, and y the target. The MSE at the top of each executed iteration is appended to `loss_history` — one float per iteration, recorded before the update it describes — and training stops early with `converged_ = True` as soon as two consecutive losses differ by less than `tol`. The fit exposes `coef_` expressed against the standardized features and `intercept_` in original target units, together with the stored `mean_` and `std_` it used, so `predict` re-applies the recorded standardization before the linear map and thus scores consistently across the train, validation, and test blocks of a `three_way_split`. Like `ridge_site_fit` the trainer draws no randomness of its own — no shuffling, no stochastic gradient — so its entire loss history is deterministic for fixed inputs: the same rows and hyperparameters replay the same loss curve to the last float, and the per-iteration record doubles as the divergence/plateau diagnostic that a scalar final loss cannot provide.

1.8 Verification

`tests/test_learning_eval.py` pins all of the above with fixed seeds and no mocks: disjoint-exhaustive folds with exact fold sizes, seed-determinism and seed-sensitivity of the splits, the $\lambda^* = 0$ and $\lambda^* > 0$ regimes with their table identities (mean and standard deviation recomputed from the fold table; the refit reproduced by hand), exact-zero errors for the correctly specified dynamics against a $> 2\times$ continuation blow-up for the misspecified one, manual recomputation of the one-step-ahead error, fraction clamping at both extremes, and the learning curve’s final-below-first invariant. The module carries 100% statement and branch coverage, and the validation rules of every entry point (distinct normalized sites, $k \in [2, n]$, fractions in the open unit interval, non-empty grids, minimum row counts) are each exercised.

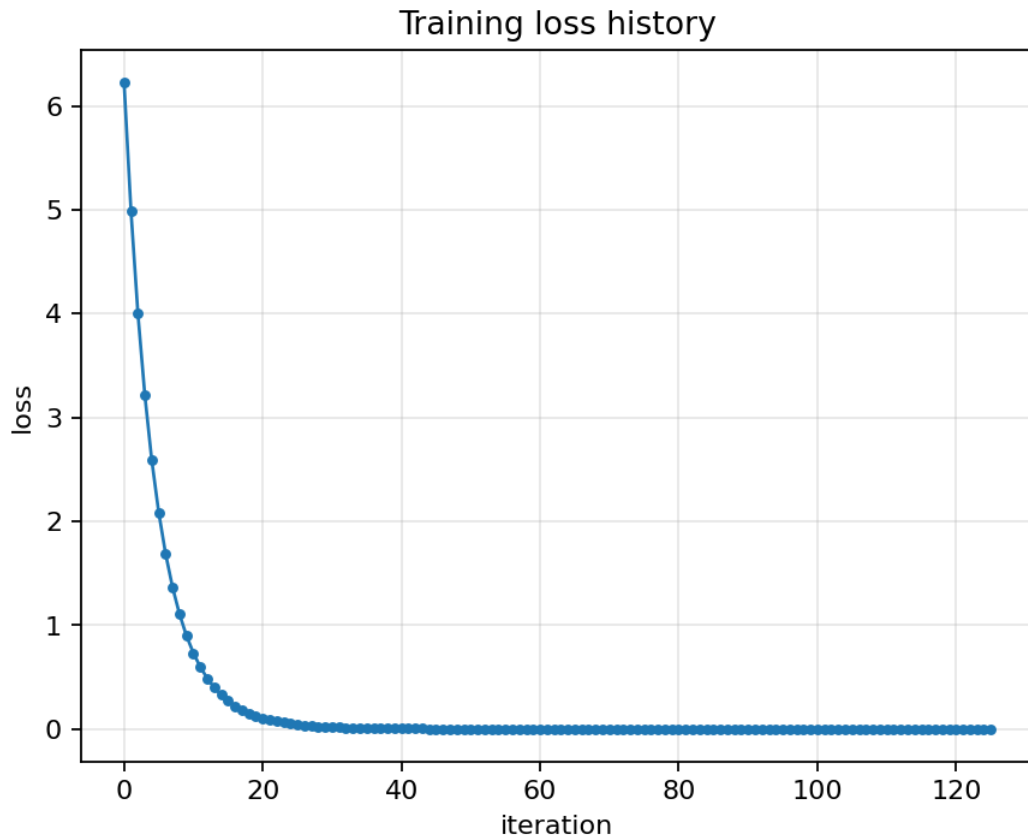


Figure 1: **Convergence of the full-batch gradient-descent trainer.** Per-iteration training loss (mean squared error, linear y -axis) of `GradientDescentTrainer` on a deterministic synthetic linear problem: a 40×3 standard-normal design matrix with true coefficients $(1.5, -2.0, 0.75)$, intercept 0.5, and Gaussian target noise $\sigma = 0.05$ (seed 0). Fitted with $\text{lr} = 0.05$, $\text{max_iters} = 300$, $\text{tol} = 1\text{e-}9$; the loss falls from 6.23 at iteration 0 to the noise-floor scale $\sigma^2 = 2.5 \times 10^{-3}$, flagged as converged after 126 iterations. Regenerate with `quadmath/scripts/learning_gallery.py`.

1.9 Cross-References

- Coordinate foundations and the twelve-around-one shell: [Quadray Methods](#)
- The learner being evaluated: [Static IVM Field Learning](#)
- The dynamics being identified: [IVM Lattice Dynamics](#)
- Enumeration and search infrastructure: [Lattice Tooling](#)